



LeetCode 35: Search Insert Position

1. Problem Title & Link

Title: LeetCode 35: Search Insert Position

Link: <https://leetcode.com/problems/search-insert-position/>

2. Problem Statement (Short Summary)

Given a **sorted array** of distinct integers `nums` and a target value `target`:

- If target exists, return its index.
- If target does not exist, return the index where it should be inserted to maintain sorted order.

You must solve it in:

$O(\log n)$

time complexity.

Example

Input:

`nums = [1,3,5,6]`

`target = 5`

Output:

2

Because:

`nums[2] = 5`

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,3,5,6]`

`target = 5`

Output:

2

Example 2

Input:

`nums = [1,3,5,6]`

`target = 2`



Output:

1

Explanation:

$1 < 2 < 3$

Insert at index:

1

Example 3

Input:

nums = [1,3,5,6]

target = 7

Output:

4

Insert at end.

Example 4

Input:

nums = [1,3,5,6]

target = 0

Output:

0

Insert at beginning.

4. Constraints

$1 \leq \text{nums.length} \leq 10^4$

$-10^4 \leq \text{nums}[i] \leq 10^4$

nums contains distinct values

nums is sorted

Important:

Must solve in $O(\log n)$

5. Core Concept (Pattern / Topic)



Binary Search

Secondary Topics:

- Lower Bound
- Search Space Reduction

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Traverse array.

```
for i in range(len(nums)):
```

```
    if nums[i] >= target:
```

```
        return i
```

If not found:

```
return len(nums)
```

Works.

Problem

Complexity:

$O(n)$

Question expects:

$O(\log n)$

Key Observation

Array is sorted.

Example:

[1,3,5,6]

Target:

2

Middle:

5

Since:

$2 < 5$

Target must be on left side.

Discard right half.

Important Insight

Even if target doesn't exist,

Binary Search naturally converges to the correct insertion position.

**Example**

Input:

nums = [1,3,5,6]

target = 2

Step 1

L M R

1 3 5 6

↑

3

Since:

 $2 < 3$

Move:

right = mid - 1

Step 2

L R

1

Middle:

1

Since:

 $2 > 1$

Move:

left = mid + 1

Loop ends.

Now:

left = 1

This is exactly where:

2

should be inserted.

7. Visual / Intuition Diagram (ASCII Diagram)



Example:
 nums = [1,3,5,6]

target = 2

Initial:

L M R

1 3 5 6

↑

Target smaller.

Move left.

L M R

1

↑

Target bigger.

Move right.

Stop:

left = 1

Insert here.

1 | 3 5 6

↑

8. Pseudocode (Language Independent)

```

left = 0
right = n - 1

while left <= right

    mid = (left + right) / 2

    if nums[mid] == target

        return mid

    else if nums[mid] < target

        left = mid + 1

    else

        right = mid - 1

return left
  
```

9. Code Implementation



Python

```
class Solution:
    def searchInsert(self, nums, target):

        left = 0
        right = len(nums) - 1

        while left <= right:

            mid = (left + right) // 2

            if nums[mid] == target:
                return mid

            elif nums[mid] < target:
                left = mid + 1

            else:
                right = mid - 1

        return left
```

Java

```
class Solution {

    public int searchInsert(
        int[] nums,
        int target) {

        int left = 0;
        int right = nums.length - 1;

        while(left <= right) {

            int mid =
                left + (right - left) / 2;

            if(nums[mid] == target)
                return mid;

            else if(nums[mid] < target)
                left = mid + 1;

            else
                right = mid - 1;
        }

        return left;
    }
}
```



```
}  
}
```

10. Time & Space Complexity

Time Complexity

$O(\log n)$

Each iteration removes half of the search space.

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Returning:

right

instead of:

left

Insertion position is:

left

Mistake 2

Using Linear Search.

Violates:

$O(\log n)$

requirement.

Mistake 3

Not handling:

Insert at beginning

Insert at end

cases.

Edge Cases

Insert at Beginning

Input:



[1,3,5,6]

target = 0

Output:

0

Insert at End

Input:

[1,3,5,6]

target = 7

Output:

4

Existing Element

Input:

[1,3,5,6]

target = 5

Output:

2

12. Detailed Dry Run

Input:

nums = [1,3,5,6]

target = 2

Left	Right	Mid	nums[mid]	Action
0	3	1	3	Go Left
0	0	0	1	Go Right

Loop ends.

left = 1

Return:

1

13. Common Use Cases (Real-Life / Interview)



Database Indexing

Finding where a new record should be inserted.

Ranking Systems

Determine position in sorted leaderboard.

Inventory Management

Find insertion point in sorted stock lists.

Interview Purpose

Tests:

Binary Search

Boundary Handling

Lower Bound Logic

14. Common Traps (Important!)

Trap 1

Thinking this is different from Binary Search.

Actually:

Binary Search

+

Return insertion point

Trap 2

Returning:

right + 1

Sometimes works.

Less intuitive than:

left

Trap 3

Forgetting that array has:

Distinct values

No duplicates to worry about.



15. Builds To (Related LeetCode Problems)

LC 704 - Binary Search



LC 35 - Search Insert Position



LC 69 - Sqrt(x)



LC 278 - First Bad Version



LC 367 - Valid Perfect Square



LC 34 - Find First and Last Position

These are classic Binary Search progression problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Linear Search	$O(n)$	$O(1)$	Poor
Binary Search	$O(\log n)$	$O(1)$	Best

Approach 1: Linear Search

Find first position where:

$\text{nums}[i] \geq \text{target}$

Easy.

But not optimal.

Approach 2: Binary Search (Chosen)

Uses sorted property.

Most efficient.

17. Why This Solution Works (Short Intuition)

Binary Search repeatedly narrows the search space.

If the target is found, return its index.

If not found, the pointer left ends exactly at the position where the target should be inserted to maintain sorted order.

18. Variations / Follow-Up Questions



Follow-Up 1

Find first occurrence of target.

Related:

LC 34

Follow-Up 2

Find last occurrence.

Related:

LC 34

Follow-Up 3

Count occurrences.

Use two binary searches.

Follow-Up 4

Insert target into sorted array and return new array.

Interview Takeaway

The key pattern is:

Binary Search

+

Lower Bound

Whenever you hear:

Sorted Array

Find Position

Insert Position

Nearest Position

immediately think:

Binary Search

This problem is one of the most important stepping stones toward advanced Binary Search questions like First Bad Version, Search Range, and Binary Search on Answer.